OPERATION KIVOTOS · SECTOR 비교 트랙 · 작업 구조 개념도

몬스터 스폰 파이프라인 — 5단계, 지금 무엇을 해야 하는가

각 단계마다 **지금 있는 것 · 무엇이 문제인가 · 무엇으로 바꾸는가**를 그림으로 고정한다. 수치는 전부 `SpawnerData.json`과 `BackStreetMap.prefab`을 직접 파싱한 실측이다.

읽는 법
파랑 = 지금 정상 동작 중 · **주황** = 문제 지점 · **초록** = 목표 상태 · **보라** = 지금 하지 않고 미루는 것. 실선 화살표는 **직접 호출**, 점선은 **이벤트·비동기**다. ★ 표시는 **측정 결과를 바꾸는 항목**이다.

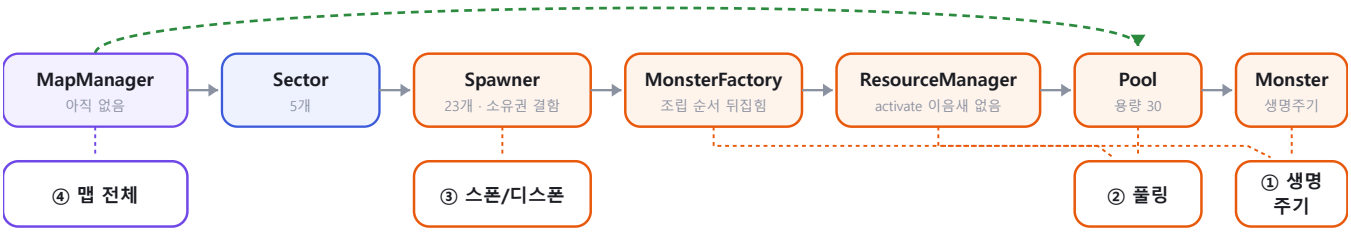
0 전체 지도 — 5단계가 파이프라인 어디에 붙는가

단계 번호는 작업 순서가 아니라 **관심사의 층**이다. 실제 착수 순서는 마지막 쪽에 있다.

① 스폰 파이프라인 전체와 5단계의 담당 구역

획득 축 — 스폰마다 (고빈도). 아래 체인이 그 경로다

용량 축 — 맵 진입·이탈에 1회 · 지금 없는 화살표



⑤ 방법론 비교 — 위 파이프라인이 전부 안정된 다음에야 얹히는 층

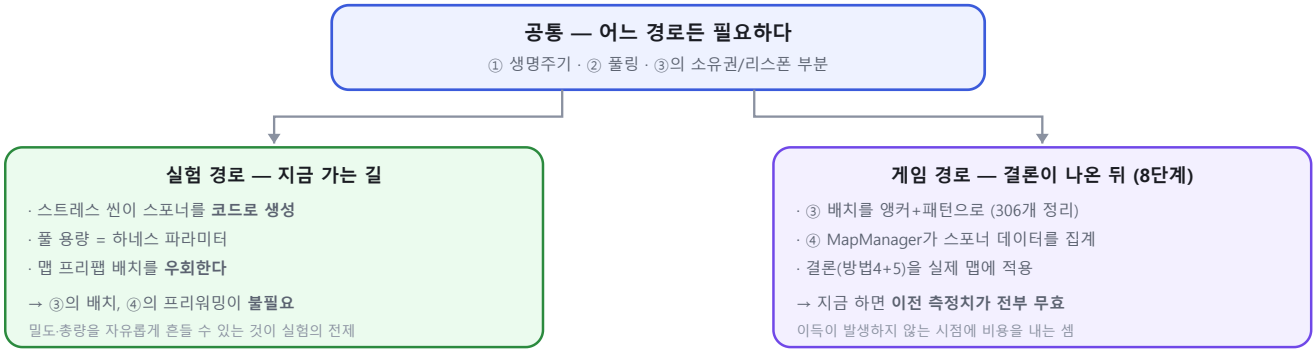
방법1 전부 켜기 · 방법2 구역(현재) · 방법3 구역+예외 · 방법4 거리 밴드 · 방법5 틱 강등

다섯은 "누구를 스폰/디스폰할 것인가"를 정하는 **정책**이고, ①~④는 그 정책이 호출하는 **기계**다.

기계가 흔들리면 정책을 비교한 표가 기계의 결함을 재게 된다. ★ 이것이 ①~④를 먼저 하는 유일한 이유다.

파랑 = 정상 · 주황 = 손봐야 하는 지점 · 보라 = 아직 없는 것. 초록 점선(용량 축)이 지금 비어 있어서 그 책임이 GameScene 하드코딩과 Pool의 즉석 생성으로 흩어져 있다.

② ③·④번은 두 갈래로 쪼개진다 오늘의 결론



"게임을 제대로 만드는 순서"와 "실험을 제대로 하는 순서"는 겹치지만 같지 않다. 겹치는 부분만 지금 한다.

단계	지금	문제	언제
① 컴포넌트 · 생명주기	동작은 함	활성화 순서가 뒤집혀 `SetStat`이 `OnEnable` 뒤에 온다. `Awake` 1회 계약이 코드에 없다	지금 · 1단계
② 풀링	동작은 함	용량이 하드코딩 30인데 실제 최대 119 ★ 부족분을 조용히 즉석 생성해 **히칭 지표를 오염**	지금 · 1단계
③ 스폰 · 디스폰	스포너 23개 배치·동작	리스폰 상태가 몬스터 인스턴스에 얹혀 있어 **방법4를 표현할 수 없다** ★ 사망~디스폰 2초간 소유가 끊긴다	소유권: 2단계 배치: 8단계
④ 맵 전체	없음. GameScene 하드코딩이 대신함	용량 축의 주인이 없어 책임이 씬 코드와 `Pool.Pop` 양쪽으로 샌다	8단계 실험은 하네스가 대체
⑤ 방법론 비교	방법1·2만 존재	3·4·5 미구현. 하네스 없음	3~6단계 · 본론

1 몬스터 컴포넌트 — 무엇을 프리팹에, 무엇을 런타임에

판단 기준은 "핵심이냐 부수냐"가 아니라 **인스펙터에서 조정할 데이터가 있는가**다.

③ 컴포넌트 소유 — 두 층과 그 경계의 근거

프리팹에 있는 것

인스펙터 데이터가 있거나, 구조를 판별하는 것

- MonsterStat
- NavMeshAgent
- Animator
- Collider
- List<AbilityData>
- Poolable ★

★ Poolable은 취향이 아니라 **구조적 강제**다.
ResourceManager가 원본 프리팹의 이 컴포넌트로 풀 경유 여부를 판정한다.

런타임에 붙는 것

SerializeField가 0개 — 인스펙터에 띄울 것이 없다

- Health — HP·피격·사망·IDamageable
- MonsterReward — 경험치·크레딧·드랍
- DamageNumberPresenter — 데미지 토스트

셋 다 EnsureOn 멱등 패턴. 풀 재사용·중복 부착에 안전하고
신규 프리팹에서 빠뜨릴 수가 없다(자가 치유).

경계를 어디에 그을 것인가 — 판단 기준

프리팹에 두면 얻는 것: **AddComponent 1회 절약**. 그런데 풀링하면 그 비용은 **인스턴스당 1회(프리워밍 중)**로 상각된다.

프리팹에 두면 잃는 것: 몬스터 프리팹 N개 수동 편집 + 신규 프리팹에서 빠뜨리면 **사일런트 버그**(보상 안 나옴·데미지 안 뜸).

→ **현행 유지가 맞다.** 바꿀 것은 컴포넌트 배치가 아니라 그 아래의 생명주기 계약이다.

이 단계에서 코드 변경은 사실상 없다. "왜 이렇게 되어 있는가"를 문서로 고정하는 것이 산출물이다.

생명주기 — 여기가 실제 작업

④ 활성화 순서 현재: 뒤집혀 있음

지금 — 팩토리가 활성화 타이밍을 통제하지 못한다

1. Pool.Pop() — 비활성 인스턴스
2. agent.Warp(pos) — agent 꺼져 있음 → 무효 의심
3. **SetActive(true)** — ResourceManager.cs:406
4. OnEnable — Recover() — MaxHp가 이전 생애 값 HP바 생성 시작 — 스탯 주입 전
5. SetStat() — 뒤늦게 덧칠 → 다시 Recover

같은 프레임이라 눈에 안 보이지만, 스폰마다 **헛일이 두 번** 돌고 순서가 취약하다.

`ResourceManager.Instantiate`가 마지막에 `SetActive(true)`를 해버려서 팩토리가 그 사이에 끼어들 자리가 없다.

⑤ 활성화 순서 목표: 활성화를 마지막으로

목표 — 조립이 끝난 뒤에야 켠다



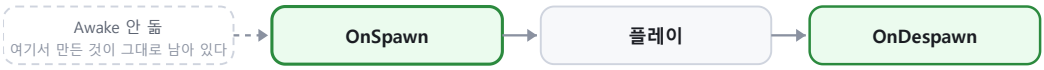
필요한 이음새는 하나 — `InstantiateAsync(..., activate:false)`. 기본값이 true라 기존 호출부(총알·이펙트)는 회귀 없음.

⑥ `Awake` 1회 계약 — 풀링이 만드는 진짜 함정

첫 생성 (프리워밍 중)



재사용 (2회차부터 — 스폰의 대부분)



따라오는 규칙 — Awake에서 만든 것은 생애를 넘어 재사용된다

지금 `ConstructBehaviorTree()`가 Awake에 있다 → **BT 노드 인스턴스가 생애를 넘어 살아남는다.**

현재는 상태가 노드가 아니라 컨트롤러 필드(_state · _target · _lastDestPosition)에 있어 우연히 안전하다. **그 우연을 규칙으로 승격시킨다.**

대안(OnSpawn마다 BT 재구축)은 스폰마다 노드 할당이라 풀링의 의미가 반쯤 사라진다. → 규칙 쪽을 택한다. (Audit M5와 같은 항목)

훅	호출 시점	담는 것
`Awake`	인스턴스당 **1회** (풀 생성 시점)	컴포넌트 캐시 · BT 구축 · `EnsureOn`. **여기 만든 것은 상태를 들면 안 된다**
`OnSpawn(ctx)`	매 스폰 · **비활성 상태**	스탯 주입 후 상태·타겟·쿨다운·BT 리셋 · HP 회복 · CTS 재생성
`OnEnable`	`SetActive(true)`가 부름	**활성 상태를 요구하는 것만** — Animator `Rebind` / `Play`. 비활성에서는 무의미하므로 여기 남는다
`OnDespawn()`	매 디스폰	구독 해제 · HP바 반납 · CTS 취소 · 스포너에 통보

이 단계의 산출물

"풀에서 꺼낸 몬스터가 이상하면 `OnSpawn` 만 보면 된다"는 상태. 지금은 리셋이 `OnEnable` · `SetStat` · `Awake` 세 곳에 흩어져 있고 그 순서를 아무도 통제하지 않는다.

❷ 풀링 — 축이 두 개인데 하나가 없다

"몬스터별 풀"은 이미 있다. 없는 것은 **그 풀들의 용량과 수명을 정하는 주인**이다.

⑦ 풀에는 성격이 다른 세 축이 있다

용량 축 — 맵 진입·이탈에 1회 (저빈도) · 지금 주인이 없다

`MapManager.Prepare()` → 집계 → `Pool.Reserve(key, count)`
`MapManager.Release()` → `Pool.Release(key)` — 인스턴스 파괴 + `refCount` 티켓 반납

실험에서는 하네스 파라미터가 이 자리를 대신한다

획득 축 — 스폰마다 (고빈도) · 지금 잘 동작함

`Spawner` → `Factory` → `ResourceManager` → `Pool.Rent(key)`

Rent는 절대 풀을 만들지 않는다 ← 새 규칙

반납 축 — 디스폰마다

`Monster.OnDespawn` → `Spawner` 통보 → `Resource.Destroy` → `Pool.Return(key)`

지금 코드에는 **용량 축이 아예 없다.** 그래서 그 책임이 위로는 GameScene 하드코딩으로, 아래로는 `Pool.Pop`의 즉석 생성으로 샜다.
= **"꺼내달라"는 요청이 "창고를 지어라"로 번역되고 있다.** 두 축을 가르는 선이 곧 이 단계의 작업이다.

⑧ 프리워밍 30 vs 실제 최대 필요치 실측 ★

막대 길이 = 개체 수 (119 기준 정규화). 회색 = 현재 프리워밍 30 · 색상 = 한 섹터 활성 시 실제 최대 필요



그 89회가 그대로 **>33ms 히칭 프레임**으로 잡힌다 — 그런데 히칭 프레임 수가 비교표의 핵심 지표다.

빗금 구간 = 풀에 없어서 게임 도중 새로 만들어지는 89마리. `PoolManager.Pop`은 이때 로그를 남기지 않는다.

⑨ 왜 이것이 "균일한 노이즈"가 아니라 순위를 바꾸는 노이즈인가

스폰/디스폰 빈도가 방법마다 극단적으로 다르다 → 풀 부족 비용도 방법마다 다르게 낀다



그래서 지표에 **스폰/디스폰 횟수**와 **스폰 1회 평균 비용**을 넣는다. 방법4가 졌을 때 방식 탓인지 구현 탓인지 표에서 갈라야 한다.

이 단계의 채택 조건

`Pop`이 풀을 즉석 생성하면 **개발 빌드 경고**를 띄운다. 그리고 **`poolMisses == 0`인 측정만 채택**한다. 이 경고가 있어야 "사전 할당이 실제로 다 덮었나"를 검증할 수 있고, 없으면 지금처럼 조용히 뚫린 채로 35조합을 다 재게 된다.

❸ 스포너 — 스폰/디스폰과 소유권

배치는 미룬다. **소유권과 리스폰만** 지금 고친다 — 그것이 방법4의 전제 조건이기 때문이다.

⑩ 소유 구간 — 지금은 사망과 회수 사이가 끊긴다

지금



사망 — 즉시 목록에서 제거 / 디스폰 연출 종료 → Push

목표



공백 구간에 섹터가 꺼지면 `DespawnMonsters()`가 그 몬스터를 모른다 → **평균·최대 활성 수 지표가 흔들린다.** 그 지표가 방법3의 유일한 증거다.

⑪ 리스폰 상태가 어디에 있는가 — 방법4가 성립하려면 여기가 바뀌어야 한다 ★

지금 — 몬스터 인스턴스에 얹혀 있다



리스폰 예약이 `MonsterSpawner.cs:176`의 람다에 걸리고,
`BMC.cs:85`의 `OnDespawned = null`이 그것을 지운다.

→ **디스폰하면 리스폰 예약도 같이 사라진다.**

목표 — 스포너가 포인트별로 소유



몬스터 인스턴스와 **완전히 독립**이므로
밴드 밖으로 나갔다 돌아와도 상태가 보존된다.

→ **"화면 밖 세계가 멈추지 않는다"가 성립.**

이것은 리팩터가 아니라 **방법4의 구현 판단 4번을 가능하게 만드는 작업**이다.
지금 코드로는 "디스폰돼도 리스폰 타이머는 스포너가 소유"를 **표현할 방법 자체가 없다** → 방법4를 문서대로 만들 수 없다.

지금 하지 않는 것 — 배치 리팩터

프리팹 자식 34개 중 **17개만 사용**(306개가 죽은 오브젝트)이고 `pointIndex`가 하이어라키 순서에 결합돼 있다. 앵커+패턴으로 바꾸면 둘 다 사라진다 — **그러나 실험 경로에는 필요 없다.** 스트레스 씬이 스포너를 코드로 생성해 배치를 우회하고, 실험 중에 배치가 바뀌면 그 전 측정치가 전부 무효가 된다.

최소 조치 둘만 지금: `pointIndex` 초과 시 조용한 `continue`에 경고 · 리젠 딜레이 지터(현재 전부 `5.0`이라 리젠이 몰린다).

❹ 맵 전체 스포너 관리 — 새 레이어의 유일한 존재 이유

`MapManager ⇔ Sector ⇔ Spawner ⇔ Factory ⇔ Pool` 체인은 **이미 있다.** 새로 만들 값어치는 하나뿐이다.

⑫ 이미 있는 것과, 정말 없는 것



↑ 이 체인은 전부 존재하고 정상 동작한다. 스포너가 풀을 직접 잡지 않고 팩토리를 경유하는 것도 이미 맞다.

없는 것은 이 화살표 하나 — 중간을 거치지 않는 용량 축

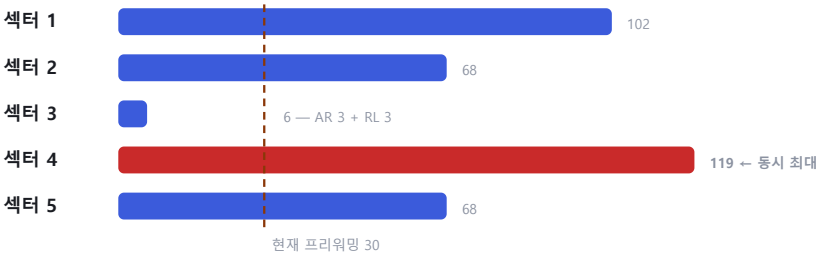
맵의 스포너 데이터를 전수 집계 → key별 동시 최대치 산출 → Reserve

그것이 지금 GameScene에 `CreatePool(prefab, key, 30)` ×3으로 손으로 박혀 있다

스포너는 획득 축만 알고 용량 축은 모른다. 이 비대칭이 의도된 것이라야 스포너를 몇 개로 늘리든 용량 결정이 한 곳에 남는다.

⑬ 섹터별 몬스터 분포 실측 — 집계가 자동이어야 하는 이유

한 번에 한 섹터만 활성 (`ActivateAllSectors`는 BenchmarkTester 전용) → 동시 최대치는 가장 큰 섹터가 결정한다



이 표가 손으로 유지될 수 있나

섹터 편차가 **6 ~ 119, 20배**다.

스포너를 하나 추가·이동할 때마다 이 숫자가 바뀌는데, 지금은 아무도 다시 계산하지 않는다.

→ **집계는 코드가 해야 한다.**

부수 효과 하나 — 섹터별 편차가 이미 20배라, 스포너 내부 구성을 균일하게 가져가도 **섹터 수준에서 밀도 변량이 확보된다.**

언제 만드나

8단계. 실험 경로에서는 스트레스 씬 하네스가 이 역할을 대신하므로 지금 만들 이유가 없다. 다만 **2단계에서 이음새는 미리 뚫어둔다** — `CreatePool`의 `count`가 호출자가 계산한 값이 되게. 출처만 다르고(하네스 파라미터 / 스포너 데이터) 이음새는 같다.

❺ 방법론 비교 — 기계가 안정된 뒤에 얹는 층

다섯은 임의의 후보가 아니라 **한 축 위의 네 점 + 직교하는 하나**다.

⑭ 두 축과 다섯 방법

존재 축 — 누구를 존재하게 할 것인가 (판정 단위가 화면 크기까지 내려온다)



품질 축 — 존재하는 것을 얼마나 자주 돌릴 것인가 (직교)

방법5 — 끄지 않고 틱 등급만 강등 (60Hz / 10Hz / 2Hz) → 방법4와 동시에 적용할 수 있다. 이것이 결론의 뼈대

방법4는 **규모**를 해결하고 밀집에 무력하다. 방법5는 **밀집**을 해결하고 규모엔 여전히 O(N)이다. → 고르는 것이 아니라 **층을 나눠 겹친다.**

⑮ 측정 축 — 밀도가 본론, 총량이 확장성 증거 2026-08-22 개정

표 A — 밀도 · 본론 (총량 300 고정)

WALK ~10 · EDGE ~30 · PULL-60 · PULL-100

전부 현실 수치다. 몹 몰이는 이 장르의 정상 플레이라
"합성 상황 아니냐"는 반론이 성립하지 않는다.

답하는 질문: 무엇을 골라야 하는가

5 방법 × 4 시나리오 = 20 조합

표 B — 총량 · 확장성 증거 (WALK 고정)

200 · 1000 · 3000

절대 수치가 아니라 곡선 모양만 읽는다.
3000은 목표 시나리오가 아니라 **극한값**이다.

답하는 질문: 그 선택이 언제까지 유효한가

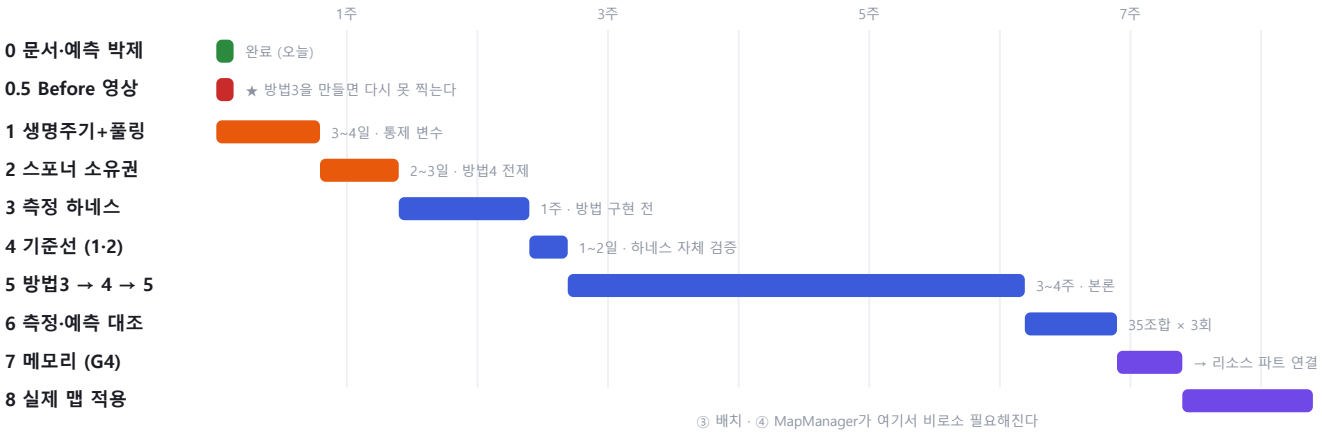
5 방법 × 3 총량 = 15 조합

여기에 **체감 지표**(팝인 횟수 · 깜빡임 · 반응 지연 · 이동 저크)가 나란히 들어간다 — 없으면 "왜 밴드 45m죠?"에 답할 수단이 없다.

착수 순서

단계 번호는 관심사의 층이었고, 아래가 실제 시간 순서다.

⑯ 순서와 대략 기간



순서의 근거 세 줄

0·0.5가 지금인 이유 — 둘 다 나중에 복구가 불가능하다. 예측은 구현하면서 결과가 보이기 시작하면 오염되고, Before 영상은 버그를 고치면 사라진다. 합쳐서 하루가 안 걸린다.
1·2가 앞인 이유 — 판단 기준은 하나다. **"이걸 고치면 다섯의 상대 순위가 바뀌나?"** 바뀌면 통제 변수다.
4를 넣은 이유 — 하네스에 버그가 있는 채로 3~5를 다 만들고 발견하면 전 조합을 다시 돌려야 한다.

Operation Kivotos · 몬스터 스폰 파이프라인 5단계 작업 구조 · 2026-08-22 · 수치 출처: `SpawnerData.json` / `BackStreetMap.prefab` 직접 파싱 · 상세 문서: `docs/MonsterSimulation/Comparison/07_Prerequisites.md`